COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	Sep 12, 2026	**DATE SUBMITTED:**	Sep 25, 2026
NAME:	Formilleza, huone Anthony	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 7	**SORTING ALGORITHMS: BUBBLE, SELECTION, AND INSERTION SORT**

1. PROCEDURE OUTPUT

Code + Console Screenshot

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sorting.h"

const int maxSize = 100;

void displayArr(int arr[]);

int main(){

    int unsortedArr[maxSize];

    srand(time(0));

    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()%100;
    }

    std::cout<<"Original Array: \n";
    displayArr(unsortedArr);

    std::cout<<"\nBubbleSort: \n";
    bubbleSort(unsortedArr, maxSize);
    displayArr(unsortedArr);

    std::cout<<"\nSelectionSort: \n";
    selectionSort(unsortedArr, maxSize);
    displayArr(unsortedArr);

    std::cout<<"\nInsertionSort: \n";
    insertionSort(unsortedArr, maxSize);
    displayArr(unsortedArr);

}

void displayArr(int arr[]){

    for(int i = 0; i < maxSize; i++){
        std::cout << arr[i] << " ";
    }

    std::cout << std::endl;
}
```

Observations	A total of 100 random numbers ranging from 0 to 99 were produced by the program. These unsorted dataset values served as the primary input for evaluating the three sorting algorithms.

Table 7-1. Array of Values for Sort Algorithm Testing

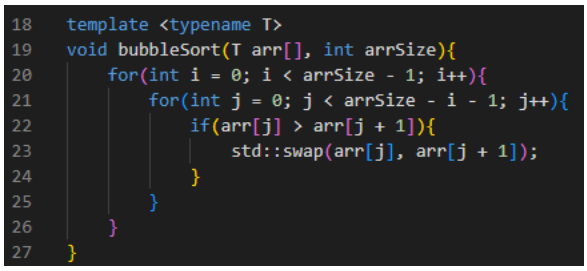
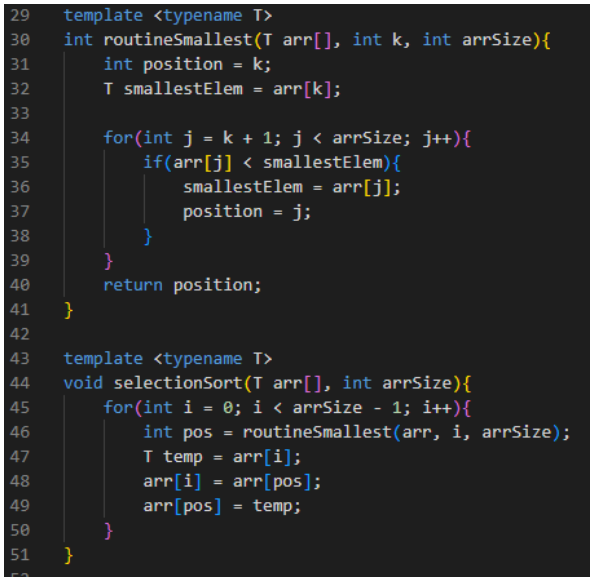
Code + Console Screenshot	
Observations	Bubble Sort successfully ordered the 100 elements in ascending order by repeatedly comparing adjacent pairs and swapping any out-of-order items. It operates with an average and worst-case time complexity of $O(n^2)$.

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	

Observations	Selection Sort successfully organized the 100 elements into ascending order by repeatedly identifying the smallest unpositioned value and placing it in its proper location. Its time complexity remains O(n²) across best, average, and worst scenarios.

Table 7-3. Selection Sort Algorithm

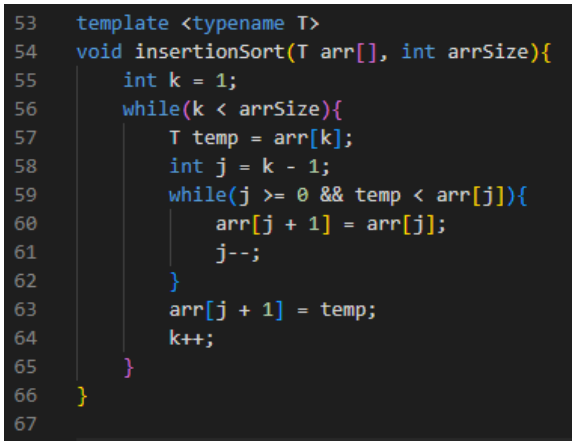
Code + Console Screenshot	
Observations	Insertion Sort successfully arranged all 100 values in ascending order by inserting each element into its proper position. Its time complexity is O(n²) in the average and worst cases, while its best case is O(n) when the array is already sorted.

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Code

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sorting.h"

const int maxSize = 100;

void displayArr(int arr[]);

int main()
{
    int votes[maxSize];

    srand(time(0));

    for(int i = 0; i < maxSize; i++)
    {
        votes[i] = rand() % 5 + 1;
    }

    std::cout << "Original Votes:\n";
    displayArr(votes);

    selectionSort(votes, maxSize);

    std::cout << "\nSorted Votes:\n";
    displayArr(votes);

    int vote1 = 0;
    int vote2 = 0;
    int vote3 = 0;
    int vote4 = 0;
    int vote5 = 0;

```

```
for(int i = 0; i < maxSize; i++)
{
    if(votes[i] == 1)
        vote1++;
    else if(votes[i] == 2)
        vote2++;
    else if(votes[i] == 3)
        vote3++;
    else if(votes[i] == 4)
        vote4++;
    else if(votes[i] == 5)
        vote5++;
}

std::cout << "\nManual Count:\n";
std::cout << "Vote 1: " << vote1 << std::endl;
std::cout << "Vote 2: " << vote2 << std::endl;
std::cout << "Vote 3: " << vote3 << std::endl;
std::cout << "Vote 4: " << vote4 << std::endl;
std::cout << "Vote 5: " << vote5 << std::endl;
```

```
int count[6] = {0};

for(int i = 0; i < maxSize; i++)
{
    count[votes[i]]++;
}

std::cout << "\nCandidate Voting Results:\n";

std::cout << "Candidate 1 - Bo Dalton Capistrano: "
          << count[1] << " votes\n";

std::cout << "Candidate 2 - Cornelius Raymon Agustín: "
          << count[2] << " votes\n";

std::cout << "Candidate 3 - Deja Jayla Bañaga: "
          << count[3] << " votes\n";

std::cout << "Candidate 4 - Lalla Brielle Yabut: "
          << count[4] << " votes\n";

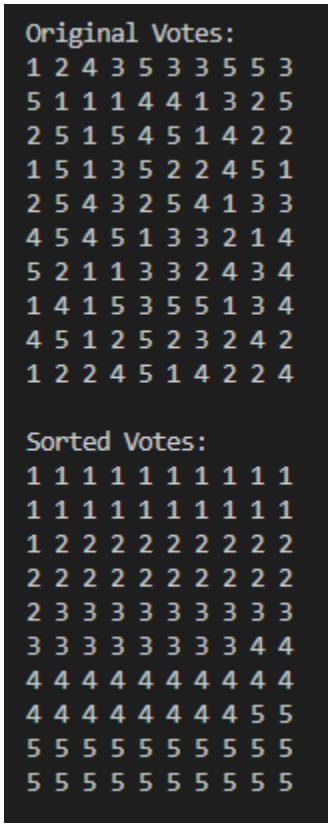
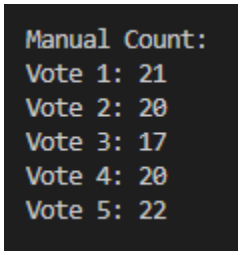
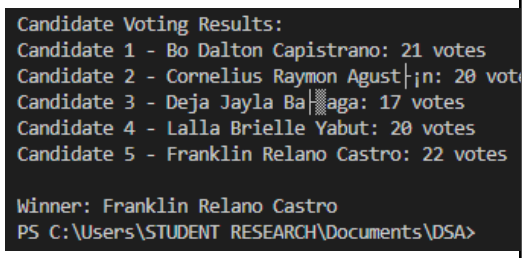
std::cout << "Candidate 5 - Franklin Relano Castro: "
          << count[5] << " votes\n";

int winner = 1;
```

```
82      for(int i = 2; i <= 5; i++)
83      {
84          if(count[i] > count[winner])
85          {
86              winner = i;
87          }
88      }
89
90      std::cout << "\nWinner: ";
91
92      if(winner == 1)
93          std::cout << "Bo Dalton Capistrano";
94      else if(winner == 2)
95          std::cout << "Cornelius Raymon Agustín";
96      else if(winner == 3)
97          std::cout << "Deja Jayla Bañaga";
98      else if(winner == 4)
99          std::cout << "Lalla Brielle Yabut";
100     else
101         std::cout << "Franklin Relano Castro";
102
103     std::cout << std::endl;
104
105     return 0;
106 }
107
108 void displayArr(int arr[])
109 {
110     for(int i = 0; i < maxSize; i++)
111     {
112         std::cout << arr[i] << " ";
113
114         if((i + 1) % 10 == 0)
115             std::cout << std::endl;
116     }
117 }
```

Pseudo Code:

START
Generate 100 random votes from 1 to 5
Display the original votes
Sort the votes using Selection Sort
Display the sorted votes
Set Vote1 to Vote5 to 0
FOR each vote
  check its value and add 1 to the matching vote counter
END FOR
Display the vote counts
Compare all vote counts
Display the candidate with the highest votes
END

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
		

Question. Was your developed vote counting algorithm effective? Why or why not?

Yes it was effective. Sorting the votes first made it simple to count how many of each value 1 to 5 appeared, and cross checking the manual count against the count array showed that they matched. So the algorithm correctly counted the votes and picked the right winner.

Summary of lessons learned

I kind of picked up on how bubble sort, selection sort, and insertion sort all go about putting a list in order, even though they don't really do it the same way. Like, bubble sort is mostly about swapping things right next to each other until everything seems to fit, while selection sort feels more like looking for the smallest thing left and moving it over. Then insertion sort is sort of like building up a sorted part piece by piece as you go. Also, it turned out that sorting isn't just for ordering items, since it can help out when you need to group things together to count them.

Analysis of the procedure

Testing the three sorts on the same set of 100 random values made it easy to see that they all ended up with the same final sorted array but got there in different ways. Bubble sort felt the most repetitive because it kept looping over the whole array, selection sort was more direct because it grabs the smallest value to put next, and insertion sort made the most sense to me as it is close to the process of hand sorting.

Analysis of the supplementary activity

This part showed a practical use of sorting besides just sorting. Once the votes were sorted, all the same values were grouped together, making it simple to count them. Comparison of the counts made it simple to find the highest and declare a winner.

Concluding statement / feedback

I think I did well overall in that I managed to get all three sorting algorithms working as well as the vote counting extension. My main room for improvement is with understanding how the index bounds work especially with insertion sort's inner while loop as it is easy to get an off by one error there. I plan to practice more with different array sizes and edge cases like an already sorted array to get a better grasp.
Want a second opinion?
Analyze with Clever AI Detector.
100% free to use

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Fixing my grammar**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.



I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.